

Software Design Document (SDD)

Module: tcs3472_control - RGBC Closed-Loop Control Module

Version: 1.2

Authors: Steven McClellan, Vania Leal, Leonardo García, Melanie Picen

Revised to align with tcs3472_driver SDD

Item	Description
Source files	tcs3472_control.c / tcs3472_control.h
Main purpose	Automatic gain control (AGC) and automatic exposure control (AEC) for the TCS3472 color sensor.
Main public functions	rgbc_control_init(), rgbc_control_update(), rgbc_get_raw_data()
Revision focus	Clarified layer separation, driver dependency direction, sequence diagrams, and CONTROL_SATURATED behavior.

1. Introduction

1.1 Purpose

This document describes the design and architecture of the tcs3472_control module. The module implements a two-phase closed-loop controller for the TCS3472 color sensor. It adjusts sensor gain (AGAIN) and integration time (ATIME) so that the Clear channel approaches a configured setpoint while maintaining a simple API for the application layer.

1.2 Scope

The module is a software control layer above the TCS3472 driver and the I2C driver. It does not directly manipulate sensor or MCU hardware registers. Instead, it calls tcs3472_init(), tcs3472_config(), and tcs3472_read_rgbc() from the TCS3472 driver to configure and sample the sensor. It also uses serial_printf() only for debug/status output during the control loop.

Dependency direction is one-way: tcs3472_control depends on tcs3472_driver, but tcs3472_driver does not depend on tcs3472_control. This keeps the sensor driver reusable in projects that do not need closed-loop AGC/AEC behavior.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
TCS3472	RGB color sensor with red, green, blue, and clear channels.
RGBC	Red, Green, Blue, and Clear sensor data.
AGAIN	Analog gain setting of the TCS3472 sensor: 1X, 4X, 16X, or 60X.
ATIME	Integration time register. Lower register values increase integration time and sensitivity.
AGC	Automatic Gain Control. Phase 1 of the algorithm sweeps AGAIN.
AEC	Automatic Exposure Control. Phase 2 of the algorithm searches ATIME.
Setpoint	Target Clear-channel count, configured as RGBC_CONTROL_SETPOINT = 6000.
Hysteresis	Allowed dead band around the setpoint, configured as +/-300 counts.
I2C	Inter-Integrated Circuit bus used to communicate with the TCS3472.
SDD	Software Design Document.

1.4 Requirements Traceability

Req. ID	Function	Requirement Statement
FR-1	rgbc_control_init()	The system shall validate input pointers, initialize the control structure, load default gain/time values, and initialize the TCS3472 sensor through the TCS3472 driver.
FR-2	rgbc_control_update()	The system shall execute a complete control cycle using Phase 1 AGC and Phase 2 AEC.
FR-3	rgbc_control_update()	The system shall return CONTROL_UNDEREXPOSED when 60X gain and 700 ms integration do not reach the setpoint.
FR-4	rgbc_control_update()	The system shall set is_stable when the Clear channel is inside the hysteresis band or when the best available approximation is accepted.
FR-5	rgbc_get_raw_data()	The system shall expose the last raw R, G, and B values through caller-provided pointers.
FR-6	print_status_line()	The system shall print one debug/status line for each sensor reading during the control loop.

Non-functional requirements: no dynamic memory allocation; validation of pointer arguments; all sensor access performed through the TCS3472 driver; debug output is polling/serial based; timing is implemented with a NOP delay loop based on the selected ATIME value.

2. System Architecture

2.1 High-Level Design

The module follows a layered architecture. The Application layer owns an rgbc_control_t instance and calls the public API. The control module performs the AGC/AEC algorithm. The TCS3472 driver handles actual sensor initialization, configuration, and RGBC reads. The I2C driver performs register transactions. The Serial module is separated because it is only used for debug output and is not part of the sensor-control path.

Layer	Files / Components	Description
Application Layer	main.c or higher-level application code	Calls rgbc_control_init(), rgbc_control_update(), and rgbc_get_raw_data().
Control Layer	tcs3472_control.c / .h	Implements the closed-loop AGC/AEC algorithm and stores active sensor state.
Sensor Driver Layer	tcs3472_driver.c / .h	Provides TCS3472 initialization, runtime configuration, RGBC acquisition, and power-off services.
Communication Driver Layer	i2c_driver.c / .h, I2C_RegDef_t	Transfers register data between the MCU and the TCS3472 sensor over I2C.
Debug Output Layer	serial.c / .h	Prints status lines with serial_printf(); independent from the sensor driver path.
Hardware Layer	TCS3472 sensor, MCU I2C, MCU USART	Physical sensor and communication peripherals.

2.2 Class Diagram

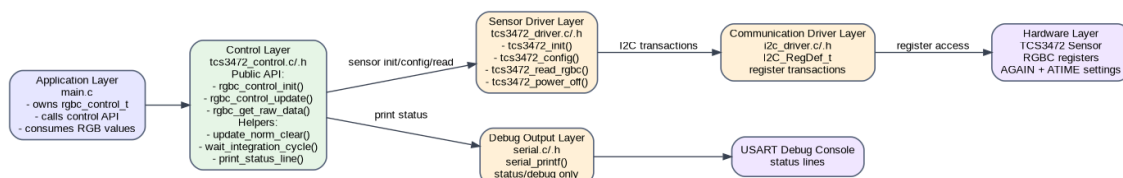


Figure 1. Class Diagram - revised tcs3472_control module layers.

2.3 Module Interfaces

Interface	Description
Public API	rgbc_control_init(), rgbc_control_update(), rgbc_get_raw_data() declared in tcs3472_control.h.
Private helpers	get_integration_cycles(), get_gain_multiplier(), gain_as_int(), update_norm_clear(), wait_integration_cycle(), print_status_line().
Sensor-driver dependencies	tcs3472_init(), tcs3472_config(), tcs3472_read_rgbc(), tcs3472_config_t, tcs3472_rgbc_t, tcs3472_gain_t, tcs3472_atime_t.
Debug dependency	serial_printf() for runtime status output only.
Indirect communication dependency	I2C_RegDef_t is passed into the control module and stored in sensor_cfg, but I2C transactions are performed by the TCS3472 driver.

3. Detailed Design

3.1 Module Behavior

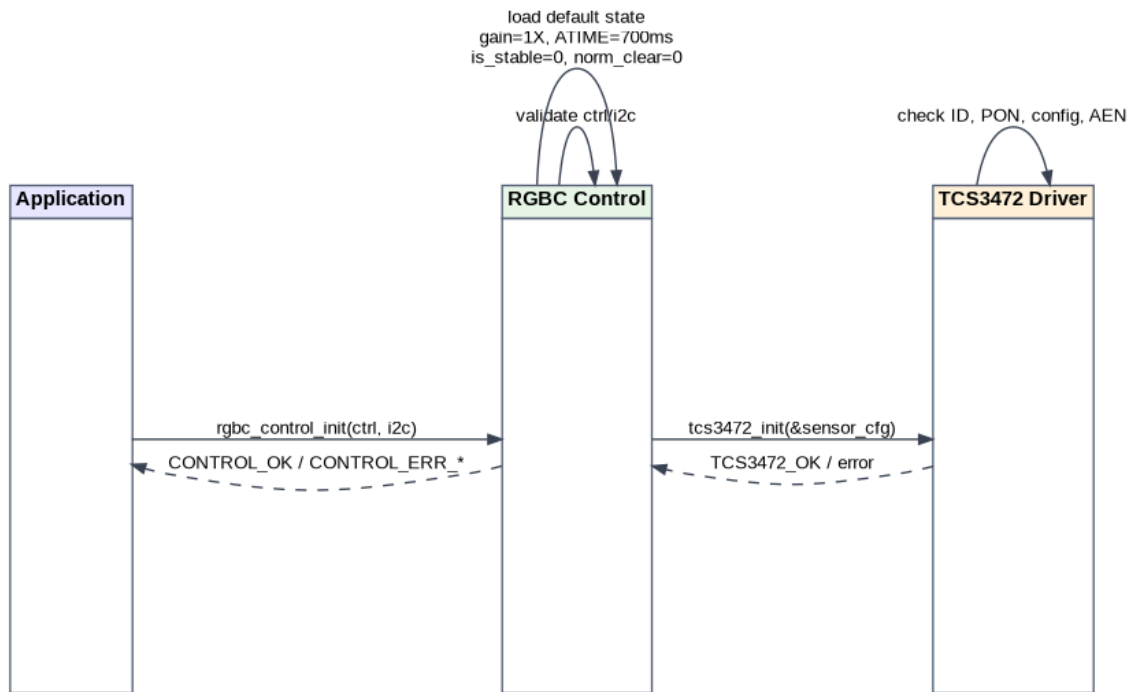


Figure 2. Sequence Diagram - rgbc_control_init.

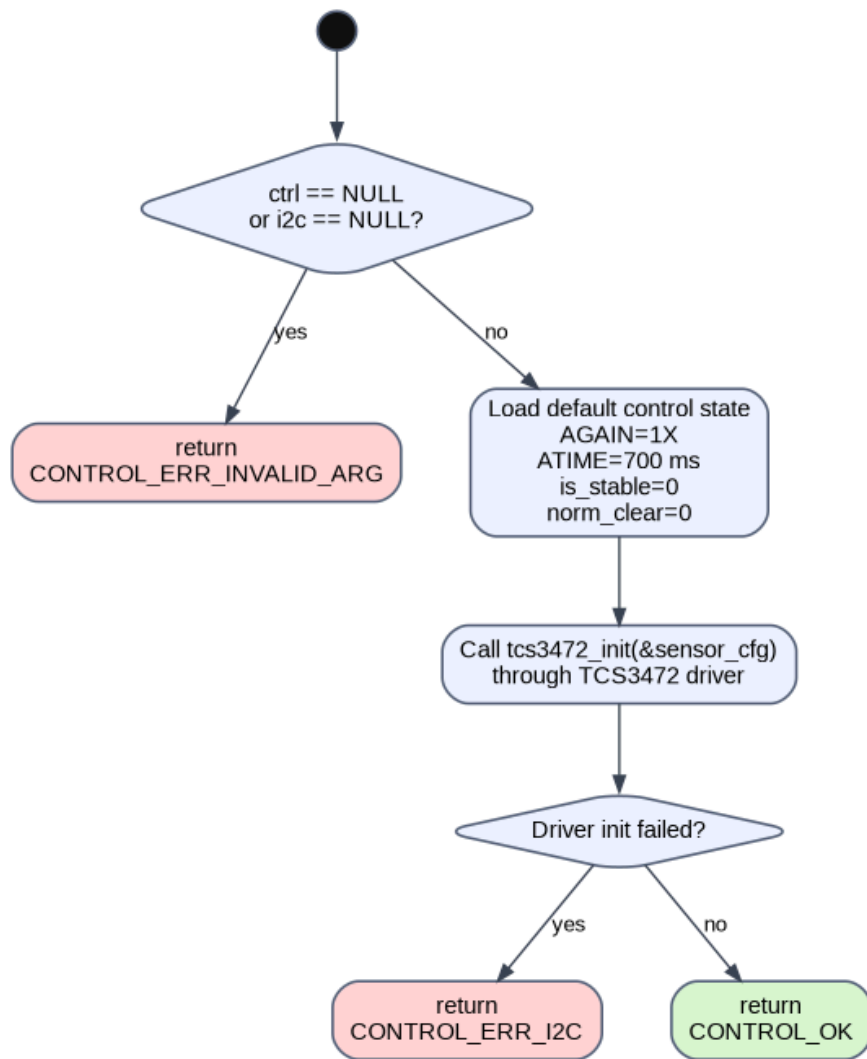


Figure 3. Activity Diagram - `rgbc_control_init`.

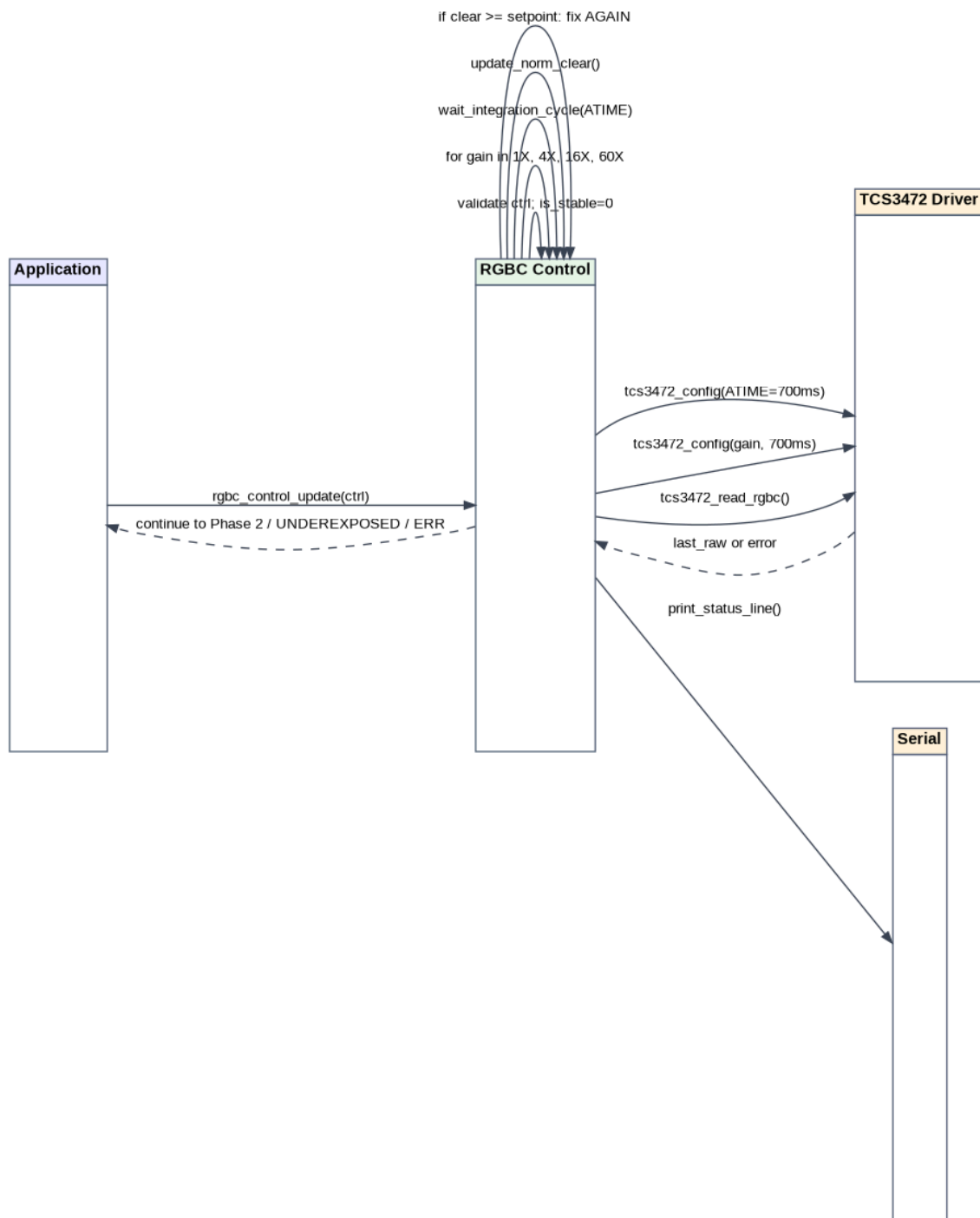


Figure 4. Sequence Diagram - Phase 1 AGC gain sweep.

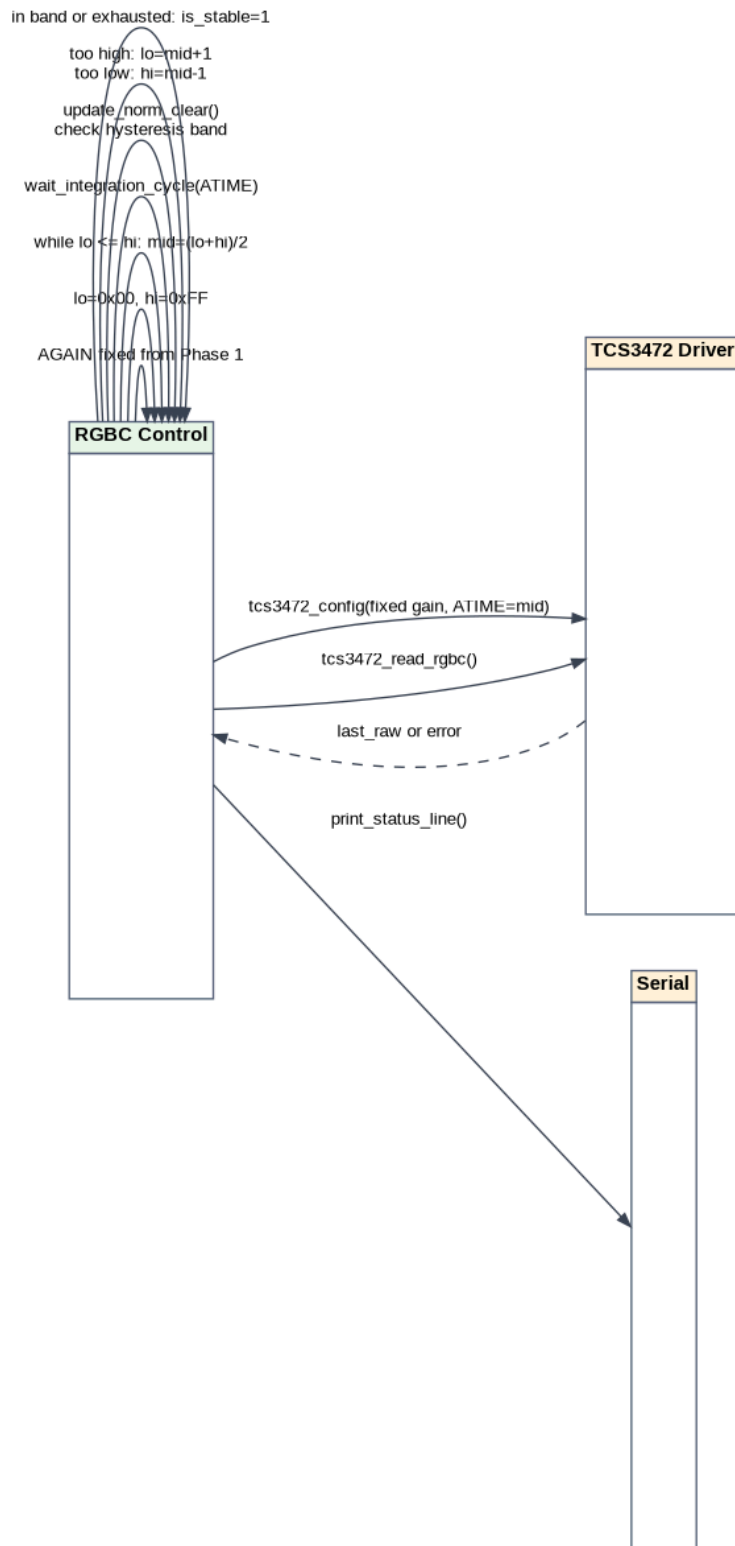


Figure 5. Sequence Diagram - Phase 2 AEC ATIME binary search.

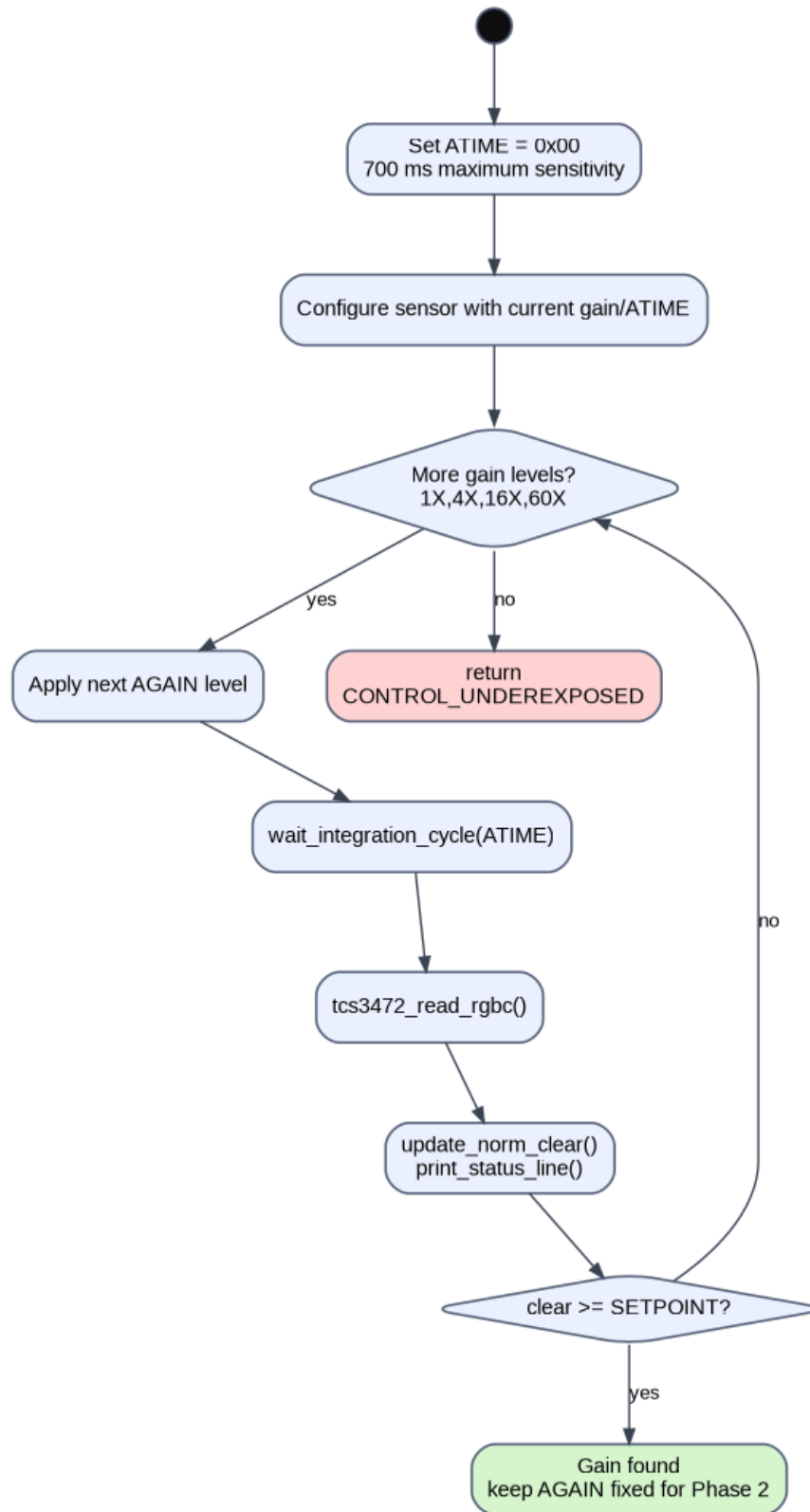


Figure 6. Activity Diagram - Phase 1 AGC gain sweep.

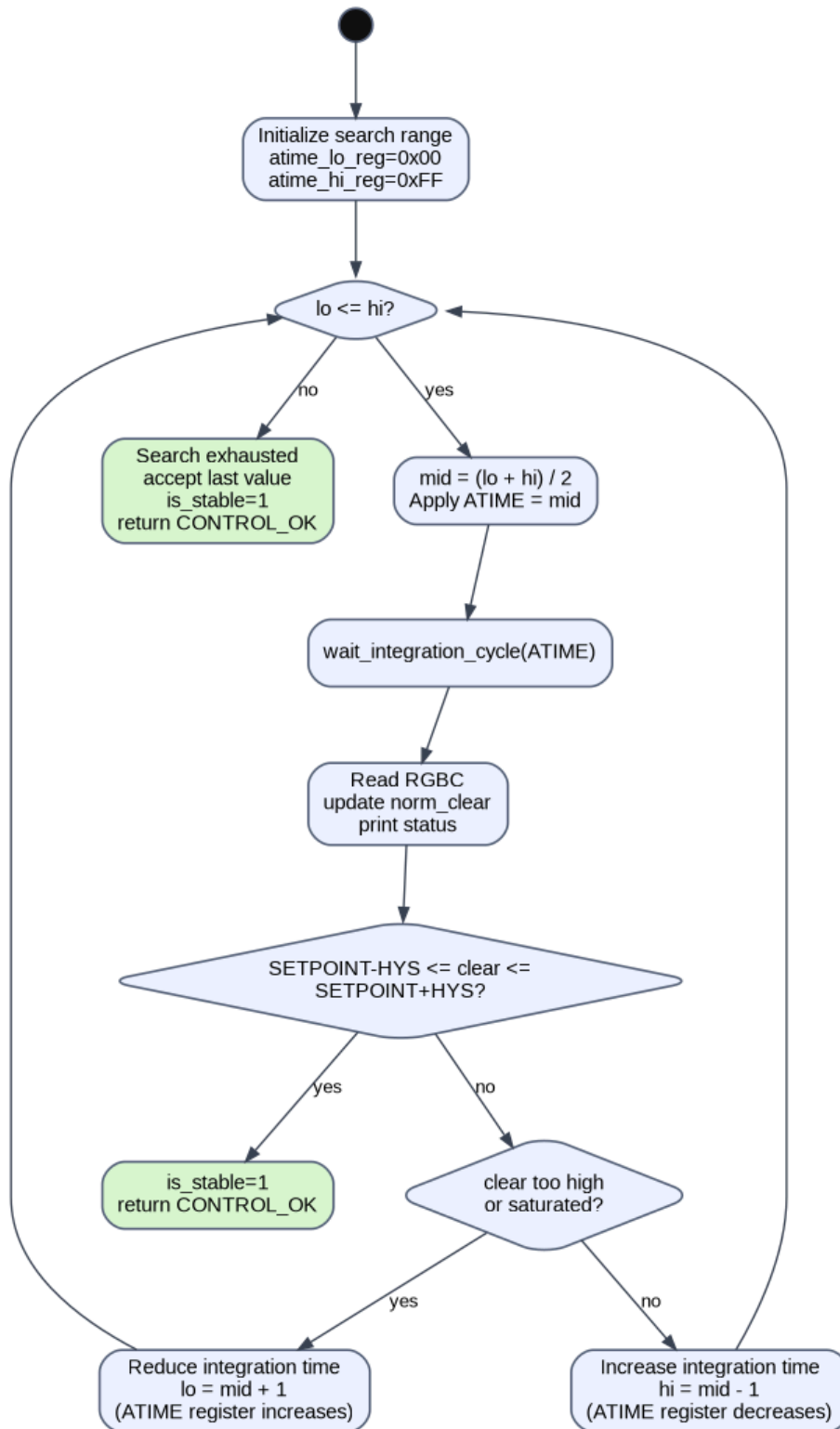


Figure 7. Activity Diagram - Phase 2 AEC ATIME binary search.

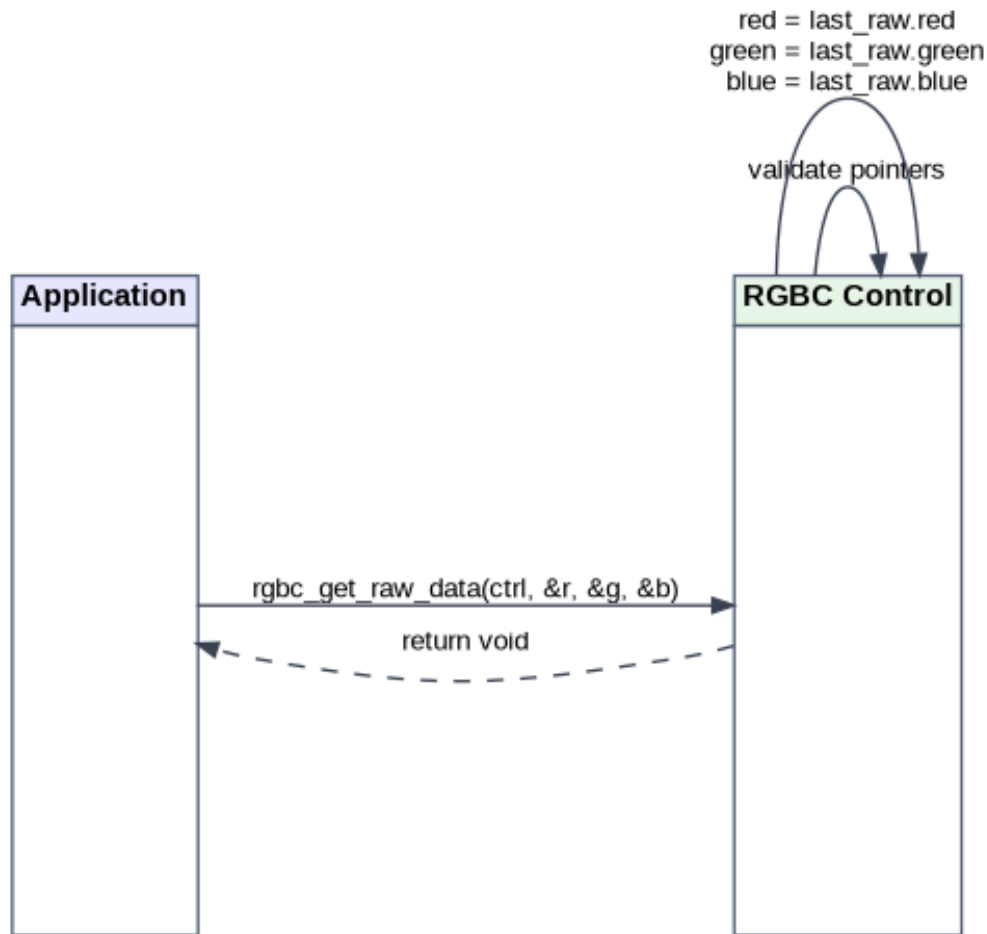


Figure 8. Sequence Diagram - `rgbc_get_raw_data`.

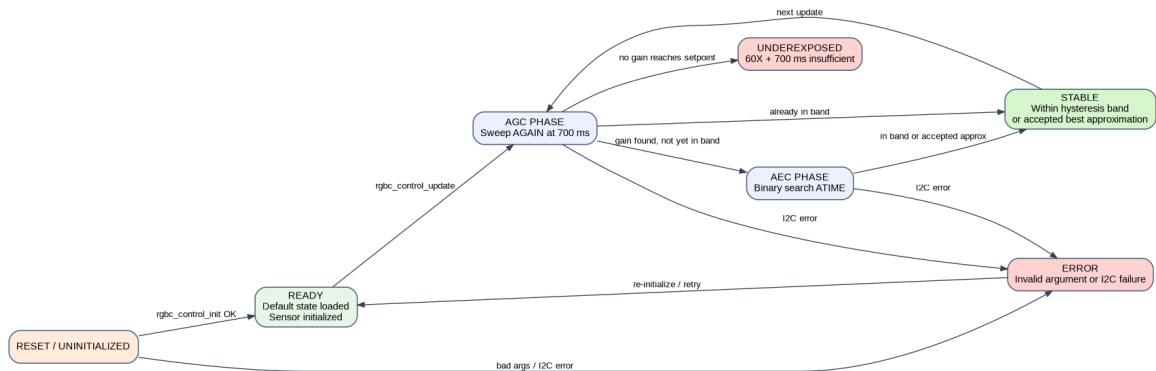


Figure 9. State Diagram - RGBC control lifecycle.

3.2 tcs3472_control Module

The control algorithm is intentionally split into two sequential phases. Phase 1 keeps ATIME at the maximum sensitivity setting, then sweeps gain from 1X to 60X. The first gain level that reaches the Clear-channel setpoint is selected. Phase 2 keeps that gain fixed and performs a binary search over the ATIME register space from 0x00 to 0xFF. Since lower ATIME register values mean longer integration time, moving the search upward shortens exposure and moving it downward lengthens exposure.

3.3 Phase 1 - AGAIN Sweep

At the start of `rgbc_control_update()`, the controller clears `is_stable` and sets `ATIME` to `TCS3472_ATIME_700MS`. It then applies gain levels in the order 1X, 4X, 16X, and 60X. After each configuration, the module waits one integration cycle, reads RGBC data, recalculates `norm_clear`, prints the status line, and checks whether `last_raw.clear >= RGBC_CONTROL_SETPOINT`. If no gain level reaches the setpoint, the function returns `CONTROL_UNDEREXPOSED`.

3.4 Phase 2 - ATIME Binary Search

Once a gain is found, the controller checks whether the current Clear value is already inside the hysteresis band. If not, it performs a binary search over the `ATIME` register. At each midpoint, it configures the sensor, waits one integration cycle, reads RGBC data, prints status, and decides whether to increase or decrease the `ATIME` register. If the Clear value is too high or saturated, the algorithm shortens integration time by increasing the lower search bound. If the Clear value is too low, it lengthens integration time by decreasing the upper search bound. If the exact band is not found before the search range is exhausted, the current implementation accepts the last value as the best approximation and returns `CONTROL_OK` with `is_stable` set.

3.5 Normalized Clear Calculation

The `update_norm_clear()` helper computes a normalized Clear value using the latest raw Clear count divided by gain multiplier and integration cycles:

```
norm_clear = last_raw.clear / (gain_multiplier * (256 - ATIME_register))
```

This normalized value is used for debug output and makes readings easier to compare across different gain and integration-time settings.

3.6 Timing Behavior

The `wait_integration_cycle()` helper converts the `ATIME` register into an approximate integration time using $(256 - \text{ATIME}) * 2.4 \text{ ms}$. It then waits using a busy loop of NOP instructions. This is simple and portable inside the current embedded setup, but it blocks the CPU while waiting.

4. Application Programming Interfaces

4.1 Data Types and Macros

Macro	Value	Notes
<code>RGBC_CONTROL_SETPOINT</code>	6000U	Ideal target value for the Clear channel.
<code>RGBC_CONTROL_HYSTERESIS</code>	300U	Dead band around the setpoint, equivalent to +/-5% of 6000.
<code>RGBC_CONTROL_MAX_COUNTS</code>	60000U	Critical saturation threshold. Used in Phase 2 high-count decision.
<code>RGBC_CONTROL_MIN_COUNTS</code>	10U	Minimum count threshold. Defined but not used in the current implementation.
<code>rgbc_control_t</code> Member	Type	Notes / Definition
<code>sensor_cfg</code>	<code>tcs3472_config_t</code>	Active TCS3472 configuration including I2C pointer, gain, and <code>ATIME</code> .
<code>last_raw</code>	<code>tcs3472_rgbc_t</code>	Last raw RGBC reading returned by the sensor driver.
<code>norm_clear</code>	float	Clear channel normalized by gain multiplier and integration cycles.
<code>is_stable</code>	<code>uint8_t</code>	1 when the controller considers the current configuration stable.

4.2 Error Codes

Code	Value	Notes / Definition
<code>CONTROL_OK</code>	0	Operation was successful. In <code>update()</code> , the control loop converged or accepted the best approximation.

CONTROL_ERR_INVALID_ARG	1	A required pointer argument was NULL.
CONTROL_ERR_I2C	2	A TCS3472 driver call failed, usually indicating communication or configuration failure.
CONTROL_SATURATED	3	Defined for future explicit saturation handling. In the current implementation, rgbc_control_update() treats saturation as a high-count condition during ATIME binary search and does not directly return this code.
CONTROL_UNDEREXPOSED	4	The sensor remains below the setpoint even at 60X gain and 700 ms integration time.

rgbc_control_init

Field	Definition
Function Name	rgbc_control_init
Syntax	control_status_t rgbc_control_init(rgbc_control_t *ctrl, I2C_RegDef_t *i2c)
Parameters (in)	i2c: pointer to the I2C peripheral instance. ctrl pointer is also validated before use.
Parameters (out)	ctrl: initialized with default sensor configuration, stability flag, and normalized value.
Return value	CONTROL_OK, CONTROL_ERR_INVALID_ARG, or CONTROL_ERR_I2C.
Description	Initializes the control structure and calls tcs3472_init() from the TCS3472 driver to verify the sensor ID, power on the sensor, configure gain/ATIME, and enable RGBC acquisition.
Constraints	ctrl and i2c must not be NULL. The function assumes the lower-level TCS3472 driver and I2C driver are correctly initialized and available.
Available via	tcs3472_control.h

rgbc_control_update

Field	Definition
Function Name	rgbc_control_update
Syntax	control_status_t rgbc_control_update(rgbc_control_t *ctrl)
Parameters (in)	ctrl: pointer to an initialized control structure.
Parameters (out)	ctrl: updates sensor_cfg, last_raw, norm_clear, and is_stable.
Return value	CONTROL_OK, CONTROL_ERR_INVALID_ARG, CONTROL_ERR_I2C, or CONTROL_UNDEREXPOSED.
Description	Runs the complete closed-loop algorithm: Phase 1 gain sweep followed by Phase 2 ATIME binary search.
Constraints	This function blocks during integration waits. It prints status lines through serial_printf(). It accepts the last value as stable when binary search is exhausted. CONTROL_SATURATED is not directly returned by the current implementation.
Available via	tcs3472_control.h

rgbc_get_raw_data

Field	Definition
Function Name	rgbc_get_raw_data
Syntax	void rgbc_get_raw_data(const rgbc_control_t *ctrl, float *red, float *green, float *blue)
Parameters (in)	ctrl: pointer to the current control state.
Parameters (out)	red, green, blue: receive the last raw RGB channels converted to float.
Return value	void.
Description	Copies the last raw red, green, and blue channel values into caller-provided output variables.
Constraints	If any pointer is NULL, the function returns without writing outputs. It does not return an error code.
Available via	tcs3472_control.h

4.3 Internal Function Definitions

Function	Syntax	Description
get_integration_cycles	static float get_integration_cycles(tcs3472_atime_t atime)	Returns 256 - ATIME register as floating-point integration cycles.
get_gain_multiplier	static float get_gain_multiplier(tcs3472_gain_t gain)	Maps enum gain values to 1.0, 4.0, 16.0, or 60.0.
gain_as_int	static int gain_as_int(tcs3472_gain_t gain)	Maps enum gain values to printable integer values 1, 4, 16, or 60.
update_norm_clear	static void update_norm_clear(rgb_control_t *ctrl)	Calculates norm_clear using last_raw.clear divided by gain and integration cycles.
wait_integration_cycle	static void wait_integration_cycle(tcs3472_atime_t atime)	Blocks for approximately one integration cycle using a NOP delay loop.
print_status_line	static void print_status_line(const rgb_control_t *ctrl, control_status_t status)	Prints RAW, normalized Clear, gain, ATIME, stability label, and status code.

5. Implementation Notes and Constraints

Topic	Note
Blocking delay	wait_integration_cycle() uses a busy NOP loop, so the CPU cannot perform other tasks during sensor integration.
Debug dependency	Every reading prints a status line through serial_printf(), which is useful for testing but can slow execution if the UART is slow.
Saturation status	CONTROL_SATURATED exists but rgb_control_update() currently treats saturation as a binary-search high-count condition instead of returning that code.
Minimum-count threshold	RGBC_CONTROL_MIN_COUNTS is defined but not used in the current algorithm.
Approximate convergence	If the binary search does not land inside the hysteresis band, the last value is accepted as the best approximation.
Modular dependency	The control module depends on the TCS3472 driver. The TCS3472 driver does not depend on the control module, so it remains reusable outside this AGC/AEC controller.

6. Future Enhancements

- Replace the NOP busy-wait with a timer-based delay or non-blocking state machine.
- Return CONTROL_SATURATED explicitly when saturation persists after exposure reduction.
- Use RGBC_CONTROL_MIN_COUNTS to detect total darkness or disconnected sensor conditions.
- Add a maximum iteration guard for the ATIME binary search for defensive programming.
- Allow the setpoint and hysteresis to be configured at runtime instead of fixed macros.
- Add unit tests for gain sweep, ATIME bisection, pointer validation, and I2C error propagation.